

Интеграционное тестирование REST API

Supertest, Express app, HTTP assertions, Authentication и test database

Результаты занятия

- ✓ Объяснять назначение integration tests для REST API
- ✓ Отделять Express app от app.listen()
- ✓ Использовать Supertest для HTTP requests
- ✓ Проверять status, headers и JSON body
- ✓ Тестировать GET, POST, PATCH и DELETE endpoints
- ✓ Передавать Authorization: Bearer token
- ✓ Использовать отдельную test database
- ✓ Подготавливать и очищать test data
- ✓ Различать unit и integration tests
- ✓ Строить повторяемый API test suite

Учебные вопросы

- 1** **Что проверяет integration test** HTTP pipeline Express целиком
- 2** **Supertest и Express app** `request(app)` без постоянного `app.listen()`
- 3** **HTTP assertions** status, headers, body и ошибки
- 4** **Authentication и protected endpoints** JWT и Authorization header
- 5** **Test database** setup, cleanup и isolation данных



Учебный вопрос 1. Что проверяет integration test

Integration test проверяет совместную работу нескольких слоёв приложения

От unit к integration

- Unit test изолирует одну функцию или Service.
- Integration test объединяет реальные компоненты.
- Для REST API это HTTP request, middleware, Router, Controller и Service.
- Если подключена test database, проверяется и слой данных.
- Цель — убедиться, что компоненты правильно взаимодействуют друг с другом.

Unit test и Integration test

Unit test

- Один модуль
- Mocks/Fakes допустимы
- Очень быстрый
- Не проверяет HTTP pipeline
- Легче локализовать failure

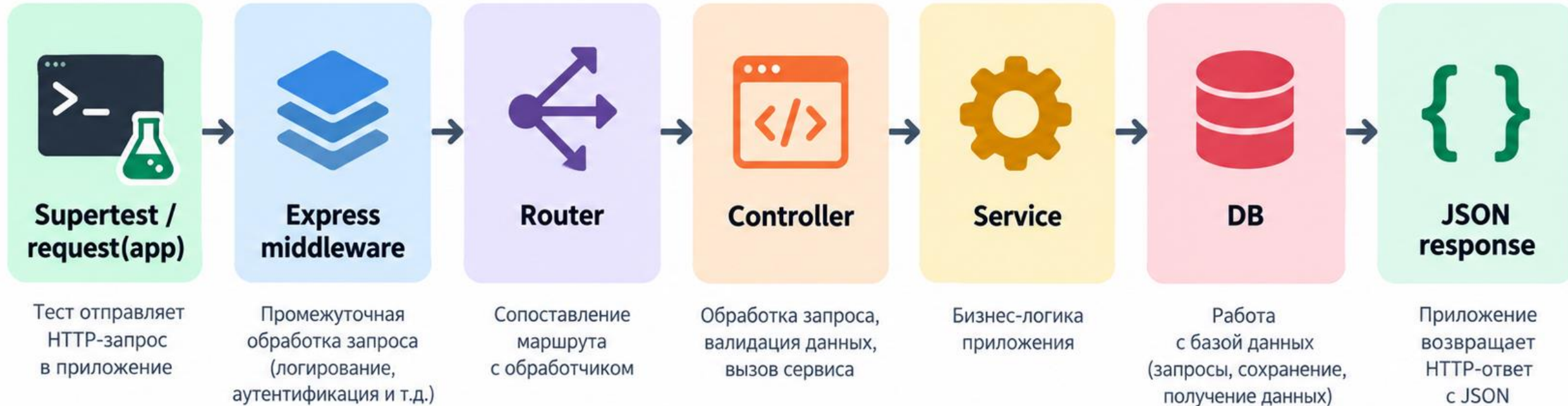
Integration test

- Несколько реальных компонентов
- Проверяет взаимодействие
- Медленнее unit
- Проверяет HTTP contract
- Failure может быть между слоями

Что проверим в Internet Shop

- GET /api/products возвращает список.
- GET /api/products/:id возвращает ресурс или 404.
- POST /api/products создаёт Product.
- PATCH /api/products/:id изменяет Product.
- DELETE /api/products/:id удаляет Product.
- Protected endpoint требует Bearer JWT.
- Validation возвращает 400 для invalid input.

Integration test: Supertest → Express middleware → Router → Controller → Service → DB → JSON response



Интеграционный тест проходит через реальные слои приложения

middleware → Router → Controller → Service → DB

API contract

- API contract — ожидаемое внешнее поведение endpoint: HTTP method, URL, status code, headers и структура response body.
- Integration tests фиксируют HTTP contract.
- Изменение внутренней реализации допустимо, пока contract сохранён.
- Это делает tests менее связанными с внутренними деталями Controller/Service.

Что НЕ надо проверять в каждом integration test

- Внутренние private variables.
- Конкретную последовательность внутренних функций, если это не часть contract.
- Каждую строку Service.
- Реализацию сторонней библиотеки.
- То, что уже надёжно покрыто unit tests.



Учебный вопрос 2. Supertest и Express app

Supertest позволяет отправлять HTTP requests прямо в Express application

Supertest

- Supertest — библиотека для HTTP assertions в Node.js, которая умеет тестировать HTTP server или Express application.
- Устанавливается как dev dependency.
- Работает вместе с Jest.
- `request(app)` формирует HTTP request.
- Если server ещё не слушает port, Supertest создаёт временный server автоматически.
- Постоянный port для test обычно не нужен.

Установка Supertest

BASH

```
npm install --save-dev supertest
```

Почему `app.listen()` нужно разделить

- `app.js` должен конфигурировать и экспортировать Express application.
- `server.js` должен только запускать `app.listen()`.
- Tests импортируют `app.js`.
- Production запускает `server.js`.
- Это позволяет тестировать приложение без занятого постоянного port.

app.js экспортирует Express application

JAVASCRIPT

```
import express from "express";
import productRoutes from "../routes/productRoutes.js";

export const app = express();

app.use(express.json());
app.use("/api/products", productRoutes);
```

Что важно

- app настраивается, но не запускает listener.
- Этот объект можно передать в Supertest.

server.js запускает приложение

JAVASCRIPT

```
import { app } from "./app.js";

const PORT = process.env.PORT ?? 3000;

app.listen(PORT, () => {
  console.log(`Server started on port ${PORT}`);
});
```

app.js vs server.js: разделяем приложение и запуск сервера

Экспортируем Express app для тестов. Запускаем listen() только в production.



app.js

Создаёт и настраивает Express application

Используется:
в production и в тестах

```
1 import express from 'express';
2 import cors from 'cors';
3 import routes from './routes/index.js';
4
5 export const app = express();
6
7 // Middleware
8 app.use(cors());
9 app.use(express.json());
10
11 // Routes
12 app.use('/api', routes);
13
14 // Error handler
15 app.use((err, req, res, next) => {
16   console.error(err);
17   res.status(500).json({ error: 'Internal Server Error' });
18 });
```



Только создание
и настройка app.
Нет app.listen()!

import { app }
from './app.js'

Тесты импортируют app из app.js



server.js

Запускает HTTP-сервер (только для production)

Используется:
только в production

```
1 import { app } from './app.js';
2 import dotenv from 'dotenv';
3
4 dotenv.config();
5 const PORT = process.env.PORT || 3000;
6
7 const server = app.listen(PORT, () => {
8   console.log(`Server running on http://localhost:${PORT}`);
9 });
10
11 // Graceful shutdown
12 process.on('SIGINT', () => {
13   console.log('\nShutting down server...');
14   server.close(() => process.exit(0));
15 });
```



Преимущества такого разделения

- ✓ Один код приложения для production и тестов
- ✓ Тесты не зависят от порта
- ✓ Быстрый запуск тестов
- ✓ Легче поддержку и масштабировать



Jest + Supertest (integration tests)

Импортируют только app, не запускают listen()

Используется:
только в тестах

```
1 import request from 'supertest';
2 import { app } from './app.js';
3
4 describe('Products API', () => {
5   it('GET /api/products returns 200', async () => {
6     const res = await request(app).get('/api/products');
7     expect(res.status).toBe(200);
8   });
9 });
```



Тесты используют
готовое приложение app.
Сервер на порту
не запускается.



Важно помнить

- ✓ app.js не должен вызывать app.listen()
- ✓ server.js используется только в production
- ✓ В тестах всегда импортируем { app } из app.js
- ✓ Supertest сам создаёт временный HTTP-сервер

Первый Supertest test

JAVASCRIPT

```
import request from "supertest";
import { describe, expect, test } from "@jest/globals";
import { app } from "../app.js";

describe("GET /api/products", () => {
  test("returns products", async () => {
    const response = await request(app)
      .get("/api/products");

    expect(response.status).toBe(200);
    expect(Array.isArray(response.body)).toBe(true);
  });
});
```

Что важно

- request(app) создаёт HTTP client для приложения.
- get() задаёт method и URL.
- response содержит status, headers и parsed body.

Что реально проходит request

- `express.json()` и другие `middleware`.
- `CORS/Helmet`, если они подключены.
- Router matching.
- Validation `middleware`.
- Authentication/Authorization.
- Controller и Service.
- Repository/Prisma, если используется test database.

request(app): временный HTTP server для integration test

Supertest сам создаёт временный сервер и не требует постоянного порта (например, 3000).



Для integration tests не нужно вызывать `app.listen()` и указывать порт!

1

Integration test (Supertest)

Тесты отправляют HTTP-запросы прямо в приложение. Supertest сам создаёт временный сервер.



product.test.js

```
1 import request from 'supertest';
2 import { app } from '../app.js';
3
4 test('GET /api/products', async () => {
5   const res = await request(app)
6     .get('/api/products')
7     .expect(200);
8   expect(res.body).toBeDefined();
9 });
```

request(app)

Supertest отправляет HTTP-запрос

Временный HTTP server



Supertest создаёт временный сервер (случайный порт, например 49152).
Не требует указания порта.

Express app



Middleware
Routes
Controllers
Services
...

HTTP response



JSON response возвращается в тест

2

Production (реальный сервер)

В production приложение запускается на фиксированном порту (например, 3000).



server.js

```
1 import { app } from './app.js';
2
3 const PORT = process.env.PORT || 3000;
4
5 app.listen(PORT, () => {
6   console.log('Server running on http://localhost:${PORT}');
7 });
```

app.listen(3000)

Запуск реального HTTP сервера на фиксированном порту

HTTP server (порт 3000)



Сервер слушает порт 3000 и принимает запросы от реальных клиентов

Express app



Middleware
Routes
Controllers
Services
...

Клиенты



Браузер, мобильное приложение, другие сервисы



Главное отличие

- В тестах порт не указывается — Supertest создаёт временный сервер.
- В production используется фиксированный порт (например, 3000).



Почему это удобно?

- Не нужно занимать порт 3000 во время тестов
- Тесты не конфликтуют между собой
- Быстрее и надёжнее запуск



Вывод

Для integration тестов достаточно экспортировать `app` из `app.js` и использовать `request(app)`.
Вызов `app.listen()` нужен только для production.



Учебный вопрос 3. HTTP assertions

Integration test проверяет внешний HTTP результат

Что проверять в response

Часть response	Пример проверки
Status	<code>expect(response.status).toBe(200)</code>
Content-Type	<code>expect(response.headers["content-type"]).toMatch(/json/)</code>
Body object	<code>expect(response.body).toEqual(...)</code>
Array length	<code>expect(response.body).toHaveLength(2)</code>
Field	<code>expect(response.body.id).toBe(15)</code>
Error message	<code>expect(response.body.error).toBe("Product not found")</code>

Проверка status и Content-Type

JAVASCRIPT

```
const response = await request(app)
  .get("/api/products");

expect(response.status).toBe(200);
expect(response.headers["content-type"])
  .toMatch(/json/);
```

GET /:id — успешный сценарий

JAVASCRIPT

```
const response = await request(app)
  .get("/api/products/15");

expect(response.status).toBe(200);
expect(response.body).toEqual(
  expect.objectContaining({
    id: 15,
    name: expect.any(String),
    price: expect.any(Number)
  })
);
```

Что важно

- `objectContaining` проверяет важную часть `contract`.
- `expect.any()` проверяет тип без привязки к конкретному значению.

GET /:id — 404

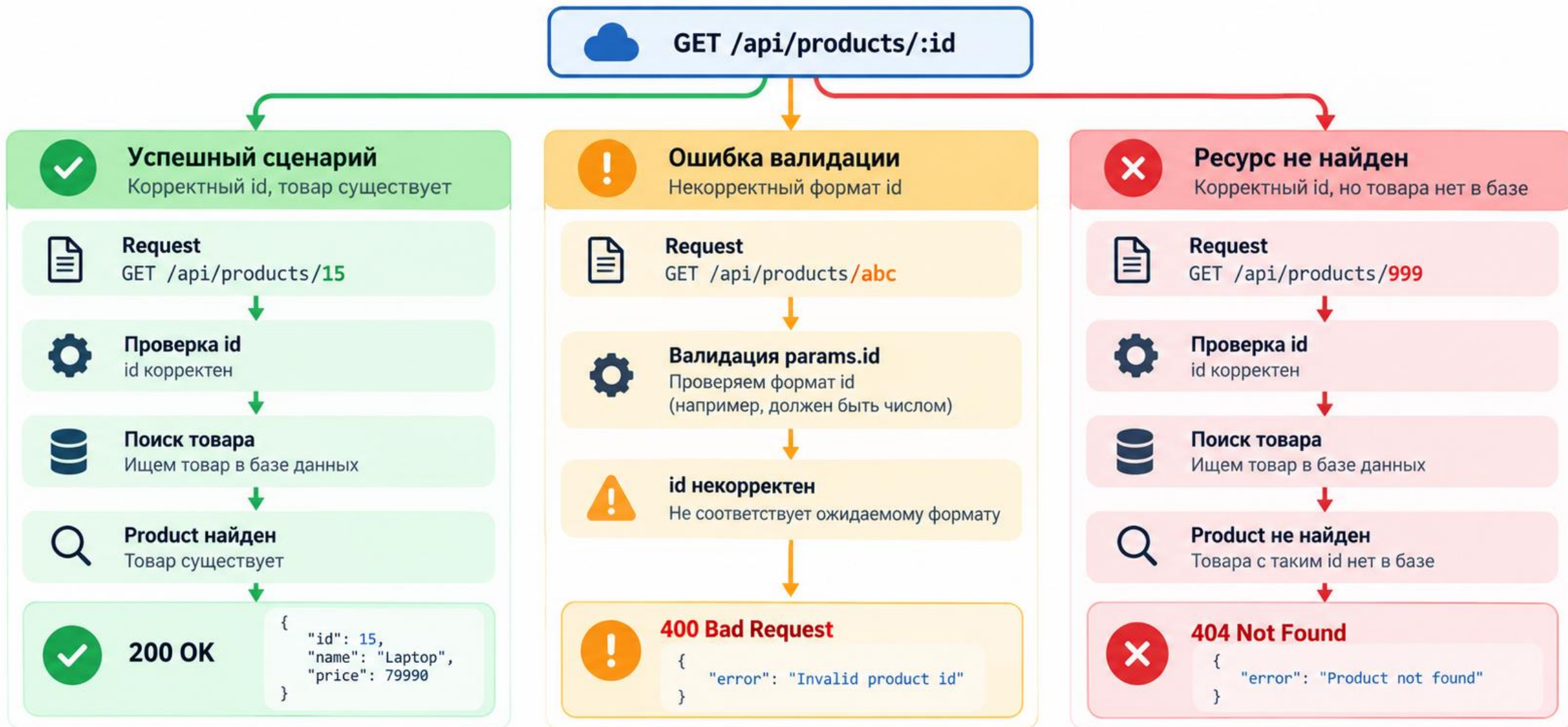
JAVASCRIPT

```
const response = await request(app)
  .get("/api/products/999999");

expect(response.status).toBe(404);
expect(response.body).toEqual({
  error: "Product not found"
});
```

Один endpoint, разные scenarios: 200 success → 400 validation → 404 not found

Один и тот же GET /api/products/:id может завершиться по-разному в зависимости от входных данных и результата поиска.



Ожидаемый результат integration test зависит от входных данных и состояния системы.

POST request через .send()

JAVASCRIPT

```
const response = await request(app)
  .post("/api/products")
  .send({
    name: "USB Hub",
    price: 12000,
    categoryId: 1,
    inStock: true
  });
```

Что важно

- send() передаёт request body.
- Supertest формирует JSON request.
- Далее Express parsing и validation работают как в приложении.

Проверка созданного ресурса

- Ожидаемый status — 201 Created.
- Response должен содержать id нового ресурса.
- Поля response должны соответствовать request и business rules.
- При test database можно дополнительно проверить, что запись действительно появилась.

Validation test — 400

JAVASCRIPT

```
const response = await request(app)
  .post("/api/products")
  .send({
    name: "X",
    price: 0,
    categoryId: 1
  });

expect(response.status).toBe(400);
expect(response.body.error)
  .toBe("Validation failed");
```



Учебный вопрос 4. Authentication и protected endpoints

Protected API нужно тестировать и без token, и с валидным token

Три базовых auth scenario

- Без token → 401 Unauthorized.
- Валидный user token, но нет роли → 403 Forbidden.
- Валидный admin token → endpoint выполняется.
- Integration test проверяет реальную цепочку `requireAuth` → `requireRole` → `Controller`.

Authorization header в Supertest

JAVASCRIPT

```
const response = await request(app)
  .post("/api/products")
  .set("Authorization", `Bearer ${adminToken}`)
  .send(productData);
```

Protected endpoint test: no token → 401; user token → 403; admin token → 201

Наглядно показать разницу Authentication и Authorization.



Authentication (Аутентификация)

Кто вы?

Проверяем, что пользователь действительно тот, за кого себя выдаёт (например, по токену).



Authorization (Авторизация)

Что вам разрешено делать?

Проверяем, есть ли у аутентифицированного пользователя нужные права (например, роль admin).



Один и тот же защищённый эндпоинт (требуется права администратора):

POST /api/admin/products (доступ только для администраторов)



1. Нет токена

Запрос без токена



Это проблема аутентификации

Система не знает, кто вы.
Пользователь не идентифицирован.



2. Токен пользователя

Валидный токен обычного пользователя



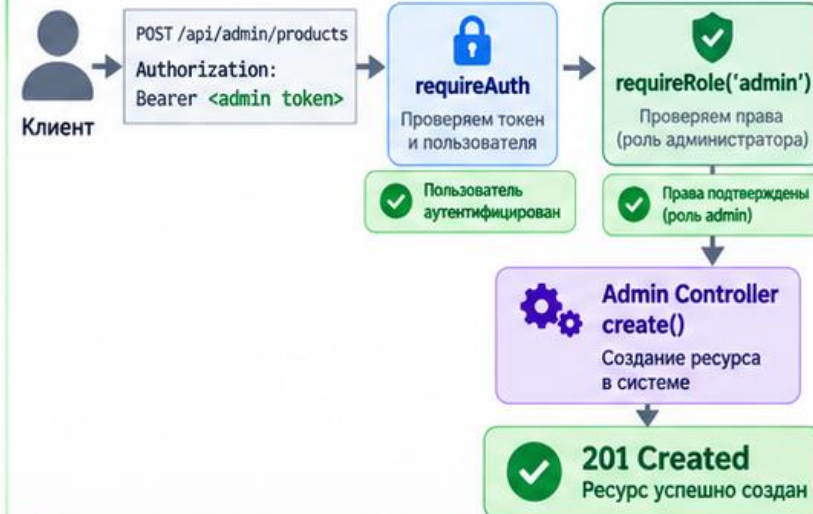
Это проблема авторизации

Пользователь аутентифицирован,
но не имеет необходимых прав
(нужна роль администратора).



3. Токен администратора

Валидный токен администратора



Доступ разрешён

Пользователь аутентифицирован
и имеет необходимые права.

Откуда взять token в test

- Вариант 1: выполнить login request в setup и получить настоящий JWT.
- Вариант 2: создать test user и подписать JWT тем же utility, что использует приложение.
- Первый вариант проверяет больше компонентов.
- Второй быстрее и удобнее для узких endpoint tests.
- Важно использовать test secret, не production secret.

Получение token через login endpoint

JAVASCRIPT

```
const loginResponse = await request(app)
  .post("/api/auth/login")
  .send({
    email: "admin@test.local",
    password: "TestPassword123!"
  });

const adminToken = loginResponse.body.accessToken;
```

Что важно

- Такой setup проверяет реальный auth flow.
- Test user должен существовать в test database.

Что проверить для protected POST

- 401 без Authorization header.
- 401 для invalid/expired token.
- 403 для недостаточной role.
- 201 для admin.
- Созданный Product действительно существует в test database.



Учебный вопрос 5. Test database

Integration tests не должны использовать development или production данные

Отдельная DATABASE_URL_TEST

- Integration tests используют отдельную базу.
- Test database можно очищать между suites.
- Migration schema должна соответствовать приложению.
- Production credentials не должны попадать в tests.
- Test data должно быть искусственным и воспроизводимым.

Пример .env.test

TEXT

```
NODE_ENV=test  
DATABASE_URL=postgresql://shop_test:college123@localhost:5432/shop_test  
JWT_SECRET=test-only-secret
```

Development DB отдельно, Test DB отдельно

Один schema contract (миграции), разные данные. Тесты не влияют на данные разработки.



Одна схема —
две базы данных.
Безопасное и изолированное
тестирование.



Prisma Migrations

(один источник правды)

```
prisma/  
├── migrations/  
│   ├── 001_init/  
│   ├── 002_add_products/  
│   └── 003_add_users/  
│   └── ...  
└── schema.prisma
```

prisma migrate



Один schema contract

- ✓ Одинаковые таблицы
- ✓ Одинаковые поля
- ✓ Одинаковые индексы
- ✓ Одинаковые отношения
- ✓ Одинаковые ограничения

применяется
к обеим БД



Development DB

(например, shop_dev)



Используется в разработке

- Реальные данные для разработки
- Можно создавать/изменять/удалять
- Используется вручную (через приложение, Prisma Studio, SQL-клиент)



Test DB

(например, shop_test)



Используется в тестах

- Отдельная база, те же миграции
- Тестовые данные (seed)
- Полностью изолирована от dev
- Создание/очистка данных в beforeEach/afterEach



.env (разные строки подключения)

```
1 # Development  
2  
3 DATABASE_URL="postgresql://user:pass@localhost:5432  
4 /shop_dev"  
5  
6 # Test  
7  
8 DATABASE_URL_TEST="postgresql://user:pass@localhost:5432  
9 /shop_test"
```



shop_dev (разработка)

products

id	name	price	stock
1	Laptop	1000	5
2	Mouse	25	100
3	Keyboard	50	50
...			

Данные разработчиков
(реальные или тестовые для ручной работы)



shop_test (тесты)

products

id	name	price	stock
1	Test Laptop	999	10
2	Test Mouse	10	20
3	Test Keyboard	20	30
...			

Только тестовые данные
(создаются скриптами/seed)



Пример в тестах (Jest + Prisma)

```
1 import { PrismaClient } from '@prisma/client';  
2  
3 const prisma = new PrismaClient({  
4   datasources: {  
5     db: {  
6       url: process.env.DATABASE_URL_TEST  
7     }  
8   }  
9 });  
10 // Тесты работают с shop_test,  
11 // не затрагивая shop_dev
```



Изоляция

Тесты не могут повредить
данные разработки.



Один schema

Одинаковые миграции и структура
таблиц в обеих базах.



Надёжность

Предсказуемые результаты тестов
в чистом окружении.



Гибкость

Легко создавать, очищать
и пересоздавать test DB.

Setup и cleanup test data

- `beforeAll` — подготовить общий `environment`.
- `beforeEach` — создать данные для конкретного `test`.
- `afterEach` — удалить изменённые данные.
- `afterAll` — закрыть соединения/`resources`.
- Tests не должны зависеть от порядка выполнения.

Очистка таблиц перед test

JAVASCRIPT

```
beforeEach(async () => {  
  await prisma.orderItem.deleteMany();  
  await prisma.order.deleteMany();  
  await prisma.product.deleteMany();  
  await prisma.category.deleteMany();  
  
  await prisma.category.create({  
    data: { id: 1, name: "Accessories" }  
  });  
});
```

Что важно

- Удаление выполняется в порядке, учитывающем FK.
- Каждый test получает известное состояние.
- Для больших suites удобно вынести seed helper.

Isolation и параллельный запуск

- Несколько test files могут работать параллельно.
- Общая DB может создавать collisions между suites.
- Учебный вариант — запуск integration suite последовательно.
- Production CI может использовать отдельные schemas/databases на worker.
- Важно, чтобы test data не конфликтовала.

Integration test lifecycle

Полный цикл одного интеграционного теста: подготовка, запрос, проверки, очистка и переход к следующему.



Каждый тест запускается в изолированном окружении с чистыми данными.

1 Setup DB

Подготовка данных (beforeEach)



```
// test/product.test.js

import { beforeEach } from
  '@jest/globals';
import { prisma } from '../src/
  db.js';

beforeEach(async () => {
  // очистка таблиц
  await prisma.product.deleteMany({});
  await prisma.user.deleteMany({});

  // тестовые данные
  await prisma.user.create({
    data: { id: 1, email: 'admin@test.com',
      role: 'admin' }
  });
});
```

Создаём чистое состояние БД и тестовые данные.

2 Request

Отправка HTTP-запроса (Supertest)



```
// отправка запроса

import request from 'supertest';
import { app } from '../src/app.js';

test('Создать продукт', async () => {
  const res = await request(app)
    .post('/api/products')
    .send({
      name: 'Laptop',
      price: 1000,
      stock: 5
    })
    .set('Authorization',
      'Bearer admin-token');
});
```

Supertest отправляет реальный HTTP-запрос в приложение (без запуска app.listen()).

3 Assertions

Проверка результата (Jest)



```
// проверки

expect(res.status).toBe(201);
expect(res.body).toEqual({
  id: expect.any(Number),
  name: 'Laptop',
  price: 1000,
  stock: 5
});

// проверяем, что данные
// действительно сохранились в БД
const product = await prisma.product
  .findUnique({ where: { id: res.body.id } });
expect(product).not.toBeNull();
expect(product.name).toBe('Laptop');
```

Проверяем статус, тело ответа и состояние базы данных.

4 Cleanup

Очистка данных (afterEach)



```
// очистка после теста

import { afterEach } from
  '@jest/globals';
import { prisma } from '../src/
  db.js';

afterEach(async () => {
  // удаляем тестовые данные
  await prisma.product.deleteMany({});
  await prisma.user.deleteMany({});
});
```

Удаляем тестовые данные, чтобы следующий тест начинался с чистого состояния.

5 Next test

Следующий тест (цикл повторяется)



```
// следующий тест

describe('Products API', () => {
  beforeEach(...);
  afterEach(...);

  test('Создать продукт', ...);
  test('Получить продукт', ...);
  test('Обновить продукт', ...);
  test('Удалить продукт', ...);
  // ... другие тесты
});
```

Jest запускает следующий тест снова: beforeEach → request → assertions → afterEach.



Повторяем для следующего теста



Изоляция

Каждый тест работает с чистыми данными.



Реальные компоненты

Тестируем реальные маршруты, middleware, контроллеры, сервисы и БД.



Надёжные результаты

Тесты не зависят друг от друга и стабильно проходят.



Автоматизация

Полный цикл выполняется автоматически в Jest.

Integration test не равен E2E

- Integration test может запускать только backend API.
- Frontend browser не участвует.
- Некоторые внешние сервисы могут быть заменены fake/test implementations.
- E2E обычно проверяет ещё более широкий пользовательский поток.
- Граница зависит от архитектуры, но термин должен быть определён в проекте явно.

Мини-практика: Product API integration suite

Написать integration tests для Product REST API через Supertest.

Задание

- GET /api/products → 200
- GET /api/products/:id → 200
- GET неизвестного id → 404
- POST без token → 401
- POST с user token → 403
- POST с admin token → 201
- POST invalid body → 400
- PATCH Product → 200
- DELETE Product → 204 или принятый в проекте status

Критерии проверки

- Tests используют request(app)
- app.listen() в tests не запускается вручную
- Используется отдельная test database
- Данные очищаются между tests
- Проверяются status и JSON response

Контрольные вопросы

1

1. Чем integration test отличается от unit test?

2

2. Для чего нужен Supertest?

3

3. Почему `app.listen()` лучше вынести в отдельный `server.js`?

4

4. Что возвращает `request(app)`?

5

5. Какие части HTTP response нужно проверять?

6

6. Как Supertest передаёт JSON body?

7

7. Как добавить Bearer token?

8

8. Почему нельзя использовать production database?

9

9. Для чего нужны `beforeEach/afterEach`?

10

10. Чем integration test отличается от E2E?

Итоги занятия

- 1 Integration tests проверяют совместную работу нескольких слоёв.
- 2 Supertest отправляет HTTP requests прямо в Express app.
- 3 app.js должен быть отделён от server.js/app.listen().
- 4 Проверяйте status, headers и body.
- 5 Тестируйте success, validation и not found scenarios.
- 6 Protected endpoints требуют tests для 401, 403 и успешного доступа.
- 7 Integration suite должен использовать отдельную test database.
- 8 Setup/cleanup обеспечивают isolation.
- 9 Tests должны быть повторяемыми и не зависеть от порядка запуска.
- 10 Следующий уровень — debugging, logging и диагностика failures.